CMDA 3634 PSA01 : OpenMP 5-center

Jason Bruno Terceros

March 27, 2024

In this write-up we consider the effect of using different parallel codes. We will be comparing the difference between Dynamic and Static parallel code with the number of threads 1,2, and 4. For these sets of tests, we will consider the set of 128 cities shown below.

1 The 5-Center Problem

Given n points $p_1, p_2, ..., p_n$ in d-dimensional space, the objective of the 5-center problem is to choose five center points c_1, c_2, c_3, c_4, c_5 from the list of n given points so that the maximum distance between any point and its closest center is minimized. Mathematically speaking, we wish to choose five center points c_1, c_2, c_3, c_4, c_5 to minimize the following cost function.

$$\text{cost}(c_1, c_2, c_3, c_4, c_5) = \max_{1 \leq i \leq n} \min(\|\mathbf{p}_i - \mathbf{c}_1\|, \|\mathbf{p}_i - \mathbf{c}_2\|, \|\mathbf{p}_i - \mathbf{c}_3\|, \|\mathbf{p}_i - \mathbf{c}_4\|, \|\mathbf{p}_i - \mathbf{c}_5\|) \quad (1)$$

In this assignment you will write and document the performance of a parallel program using OpenMP to solve the 5-center problem for two dimension points. A working sequential program to solve the 5-center problem for 2d points that uses the distance table and early dropout optimizations is included in the materials GIT repo.

2 Implementation Tasks

We were given a code that needed to be modified so that it can run parallel. This would have made it more efficient and would have reduced run time. A few examples that have helped with developing this parallel version of our provided code is omp_extreme.c, and both collatz.c and omp_collatz.c. These examples showed me how to identify how to properly determine the private thread variables that we would need through the omp parallel function. Through some hints and reading I determined the solve_5centere() is the function I would essentially need to parallel for this assignment to be successful.

Reading the function, there is a quad-loop that would need to be parallel. Here is the provided function:

```
// solve the 5-center problem exactly
// returns the minimal cost and an optimal set of 5 centers
```

```
double solve_5center (float dist_sqs[], int n,
                    centers_type* optimal, long int* num_checked)
{

    double min_cost_sq = DBL_MAX;
    *num_checked = 0;
    for (int i=0;i<n-4;i++) {
        for (int j=i+1;j<n-3;j++) {
            for (int k=j+1;k<n-2;k++) {
                for (int l=k+1;l<n-1;l++) {
                    for (int m=l+1;m<n;m++) {
                        *num_checked += 1;
                        centers_type check = { i, j, k, l, m };
                        double cost_sq = center_cost_sq
                                    (dist_sqs,n,check,min_cost_sq);
                        if (cost_sq < min_cost_sq) {
                            min_cost_sq = cost_sq;
                            *optimal = check;
                        }
                    }
                }
            }
        }
    }
    // return the minimal cost
    return sqrt(min_cost_sq);
}
```

and here is my adjusted code:

```
// solve the 5-center problem exactly
// returns the minimal cost and an optimal set of 5 centers
double solve_5center (float dist_sqs[], int n,
                    centers_type* optimal, long int* num_checked)
{

// hint said to modify this

    double min_cost_sq = DBL_MAX;
    *num_checked = 0;

    #pragma omp parallel default(none) shared(n, num_checked,
                                dist_sqs, min_cost_sq, optimal)
    {
    double thread_min_cost_sq = DBL_MAX;
    long int thread_num_checked = 0; // only one I don't know
```

```
    centers_type thread_optimal;

    #ifdef DYNAMIC
    #pragma omp for schedule(dynamic)
    #else
    #pragma omp for schedule(static)
    #endif

        for (int i=0;i<n-4;i++) {
            for (int j=i+1;j<n-3;j++) {
                for (int k=j+1;k<n-2;k++) {
                    for (int l=k+1;l<n-1;l++) {
                        for (int m=l+1;m<n;m++) {
                            thread_num_checked += 1;
                            centers_type check = { i, j, k, l, m };
                            double cost_sq = center_cost_sq
                                        (dist_sqs,n,check,thread_min_cost_sq);
                            if (cost_sq < thread_min_cost_sq) {
                                thread_min_cost_sq = cost_sq;
                                thread_optimal = check;
                            }
                        }
                    }
                }
            }
        }

        #pragma omp critical
        {
            *num_checked += thread_num_checked;
            if (thread_min_cost_sq < min_cost_sq)
            {
                min_cost_sq = thread_min_cost_sq;
                *optimal = thread_optimal;
            }
        }

    }
    // return the minimal cost
    return sqrt(min_cost_sq);
}
```

3 Testing Tasks

Once we finished implementing our code, we tested it by running two different types of schedules. This first one we ran a static schedule, Figure 1 is the ran compiler and execution for the static scheduler. While Figure 2 is our code ran as a dynamic scheduler.

1. Static

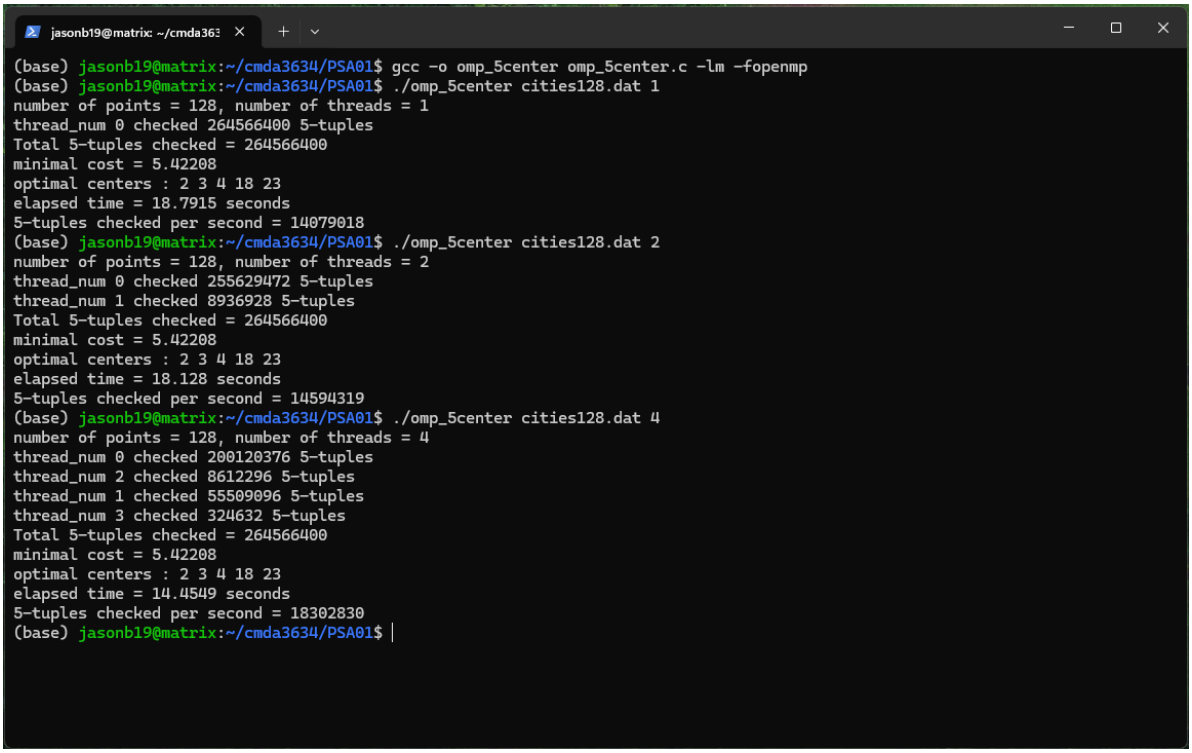

Figure 1: Shows the output of omp_5center when ran as a Static scheduler using 1, 2, and 4 threads

2. Dynamic

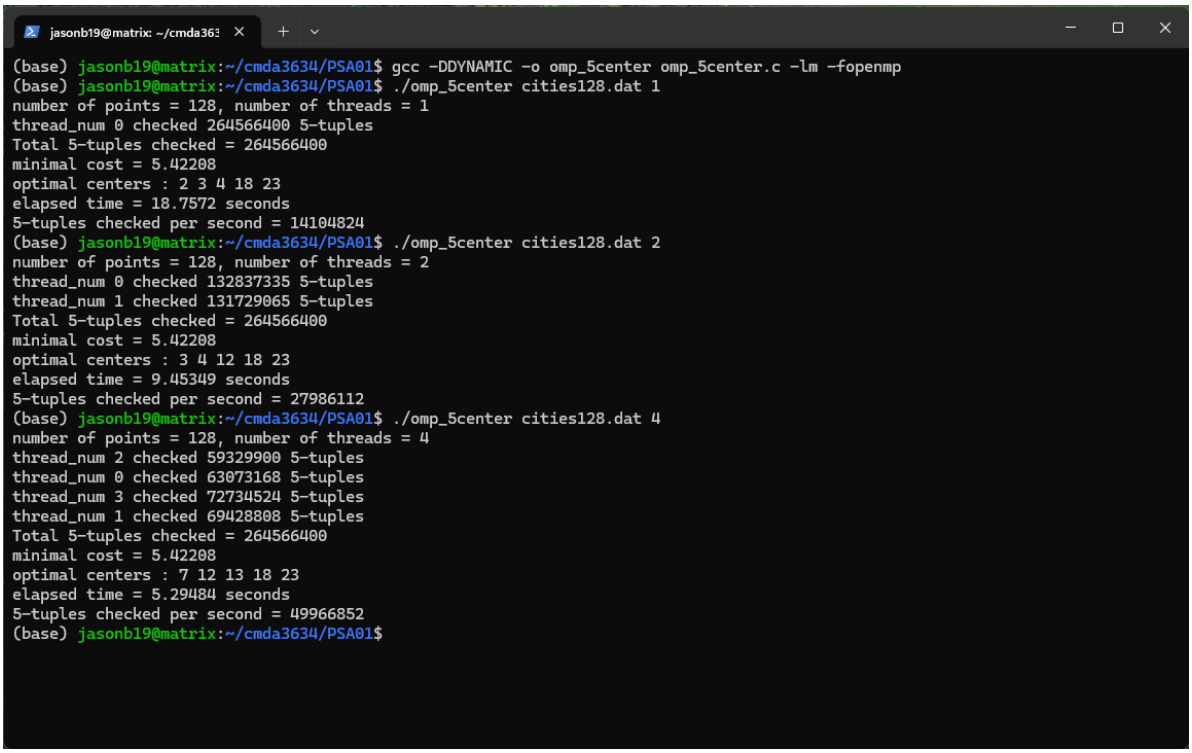

Figure 2: Shows the output of omp_5center when ran as a Dynamic scheduler using 1, 2, and 4 threads

4 Write-Up

1. Classification of each variable

   There are more **private** variables than there is shared but we can list them to the best of my ability. Some of the obvious ones are **thread_min_cost_sq**, **thread_num_checked** and **thread_optimal**. Now there are a few others that are instantiated within the `pragma omp parallel` and the for-loops. These other variables would include $i, j, k, l,$ and m. Now for my **shared** variables were more easily to identify. For example, I know that **n, num_checked, dist_sqs, min_cost_sq**, and **optimal** are all shared variables in my parallel function. We will evaluate these a little further. In order, i is a read-only shared variable, *num_checked* is read-write. This is because it is a pointer and we are incrementing it and setting it to the new value that is incremented. *dist_sqs* is the read-only shared variable. *min_cost_sq* is a read-write shared variable because in the pragma omp critical has an if-statement that reads min_cost_sq and changes it within this same logic statement if the statement is true. Finally, *optimal* is used as a pointer and it is write only shared variable. We only change it in our pragma omp critical.

2. Steps took to make thread-safe code

   I made sure to take it step by step from our proposed examples from out lectures. I first had to determine what would be my shared variables. By doing this, I identified

what would need to be in parallel code, and then read my values inside the code and outside. If it matached both of these then I knew it was shared among the inside the omp and outside. From our lecture pratice, collatz, I saw I needed to create a few iteration variables within the parallel code. The two that we used on the outside are min_cost_sq and num_checked. So I needed to make two iteration variables for my omp parallel. However, there was one that was hard to noticed because it was not straight forward, and that was the optimal pointer that we used as a parameter for our function method. Finally, with our final logical statement inside our quad-loop needs to be in an omp critical. This is where I adjusted our code so it would iterate through our inside thread variables but then at our ciritical statement, we run it through the same thread variables and adjust our actual outside varibles. Such as num_checked, min_cost_sq, and optimal. We also added a thread_num and checked tuple output within our parallel code and critical code to show the iterations and checked values.

3. Steps took to make code run efficiently in parallel

   I made sure that each thread only enters the critical region one time by placing it at the end of my for-loops (all 4 loops). That way when it is done running through all the loops (assuming simultaneously) it gets towards the end of the parallel code, then it runs the omp critical code. Thus, it only runs once at the end of the looping code.

4. Speedup Calculated

$$\text{speedup} = S_P = \frac{T_S}{T_P} = \frac{1}{(1-\alpha) + \frac{\alpha}{P}} \tag{2}$$

   We will use the second equation to determine speedup but since we ran our code we have run time and can determine it using num_threads = 1 divided by the time it took to run num_threads = 4 for the respective scheduler. When ran our static scheduler for our code at num_threads = 1 my run time is 18.7915 seconds. Same as when num_threads = 4 my run time is 14.4549 seconds. Hence, $\text{speedup}_{\text{static}} = \frac{\text{num\_threads=1}}{\text{num\_threads=4}} = \frac{18.7915}{14.4849} = 1.2973$. Now for our dynamic scheduler for my code at num_threads = 1 my run time is 18.7572 seconds. Same as when num_threads = 4 my run time is 5.29484 seconds. So, $\text{speedup}_{\text{dynamic}} = \frac{\text{num\_threads=1}}{\text{num\_threads=4}} = \frac{18.7572}{5.29484} = 3.5425$.

5. Explanation of why dynamic looping had considerable speedup

   I believe that this is depended on how much time each iteration in our code would take. There is probably some iterations that take a significant amount of time to run than others. For example, when we ran number of threads to be 1 for both static and dynamic, our elapsed time is about the same. There is a 0.0343 second difference between the two. Where our dynamic scheduler run was 0.0343 seconds faster. However, when we review our second test, where we had 2 number of threads, you can tell there is a significant time difference. Almost half the time is needed to run in dynamic than in static. This is probably because static scheduler runs in a certain sequence of parallel while dynamic scheduler runs when one thread finishes and begins the next. This is probably where we saved a lot of time. You can visually see this

when you consider the number of 5-tuples that are checked per the thread number (2). In dynamic schedule, we see that both threads evaluate approximately, 1.3×10^8. But in static we found that for the same number of threads (2) thread 1 evaluates 2.5×10^8 while the next thread runs through 8.9×10^6. We see here that this first thread number for static run time is what takes the longest amount of time. You can see this same difference in a 4 thread count for both dynamic and static. You can tell that the 5-tuples evaluated for each thread number is approximately the same. However, for static there is a clear evidence that some run for a much longer period of time. This is where we cut down on most of our time between the two different types of parallel runs.

6. Minimal cost is the same regardless of thread_count

   See, the minimal cost for our runs should all be the same. Our data is the same and thus our end evaluation should be the same for all. The main thing we are evaluating is the run time respectively for dynamic and static and the number of thread counts used. Now, I believe for the optimal centers, they change since in dynamic run time it runs based on when the respective centers have been completed. So I believe these optimal centers are different because when we added more thread counts our code split up a little differently and would run different centers based on when others finished, hence, for 2 threads it was optimal to run 3,4,12,18, and 23. But for 4 threads it was more optimal to run through 7,12,13,18, and 23 because it might have just been faster to run these in that sequence.